

TELECOM CUSTOMER CHURN ANALYSIS

An End-to-End ETL & Business Intelligence Project Using PostgreSQL and Power BI

Technical Project Report

Scope: PostgreSQL ETL + Power BI Dashboard | Machine Learning phase not implemented

Tools: PostgreSQL · Power BI (Power Query · DAX)

Date of Report: September 2026

Table of Contents

Table of Contents	2
1. Introduction	3
1.1 Project Origin and Credit	3
2. Story of Data	3
2.1 What the Data Contains	3
2.2 The Journey of the Data	4
3. Data Splitting and Preprocessing	4
3.1 Staging the Raw Data	4
3.2 Splitting Customers by Status	4
4. Pre-Analysis	5
4.1 Distribution Checks	5
4.2 Null Audit	6
5. In-Analysis	6
5.1 Cleaning Nulls Into a Production Table	6
5.2 Splitting Into Reporting Views	7
5.3 Power BI Power Query Transformations	7
6. Post-Analysis and Insights	7
6.1 Core DAX Measures	7
6.2 Live Dashboard Output	8
6.3 Key Insights From the Live Data	8
7. Data Visualizations & Charts	9
8. Recommendations and Observations	10
8.1 Business Recommendations (Data-Driven)	10
8.2 Technical & Engineering Recommendations	11
8.3 Issues to Correct in the Current Dashboard	11
8.4 Observations	12
9. Conclusion	12
10. References & Appendices	13
10.1 Reference & Credit	13
10.2 Appendix A — prod_churn Schema	13
10.3 Appendix B — Full PostgreSQL Script	13
10.4 Appendix C — Power BI DAX Measures	15

1. Introduction

Customer churn — the rate at which customers stop doing business with a company — is one of the most consequential metrics a subscription-based business can track. Acquiring a new customer is typically far more expensive than retaining an existing one, which means even small improvements in churn rate can have an outsized effect on long-term revenue. This project builds a complete churn-analysis pipeline for a telecom company: raw customer data is loaded and cleaned in a relational database, then transformed and visualized in an interactive Power BI dashboard that lets stakeholders explore churn across demographic, geographic, account, and service dimensions.

The importance of this project is threefold. First, it demonstrates a realistic, end-to-end analytics workflow — from raw CSV ingestion through database ETL (Extract, Transform, Load) to a polished, decision-ready dashboard — which mirrors how churn analysis is actually performed in industry. Second, it builds practical, transferable skills in relational database design, SQL data-quality auditing, Power Query transformation, and DAX measure authoring. Third, the resulting dashboard gives a business genuinely actionable structure: it does not just report a churn number, it breaks that number down by contract type, payment method, tenure, geography, and service usage, so that retention efforts can be targeted rather than generic.

1.1 Project Origin and Credit

This project's design, dataset structure, and step-by-step methodology were adapted from a tutorial published by the YouTube channel Pivotal Stats, titled "PowerBI End to End Churn Analysis Portfolio Project | Power BI + SQL + Machine Learning | 2024." Full credit for the original project design, dashboard layout, and analytical approach belongs to that creator; this report documents an independent implementation of that design.

Two deliberate adjustments were made to the original tutorial, and both are documented throughout this report:

- Database engine substitution: the original tutorial uses Microsoft SQL Server. This implementation uses PostgreSQL instead, since SQL Server was not available in the working environment. All SQL in this report has been written and adapted for PostgreSQL syntax (double-quoted identifiers, PostgreSQL's CREATE TABLE AS, and native COALESCE handling).
- Scope reduction: the original tutorial's title includes a Machine Learning component (a churn-prediction model). That component was intentionally not implemented in this project. This report covers only the SQL/ETL and Power BI portions of the original tutorial.

2. Story of Data

The dataset at the center of this project is a telecom company's customer record file, structured around four broad dimensions plus a churn outcome.

2.1 What the Data Contains

- Demographic attributes — Gender, Age, Married status.
- Geographic attributes — customer State.
- Account & payment attributes — Contract type, Payment Method, Tenure in Months, Monthly Charge, Total Charges, Total Refunds, Value Deal.
- Service attributes — Phone Service, Multiple Lines, Internet Service and Type, Online Security, Online Backup, Device Protection Plan, Premium Support, Streaming TV/Movies/Music, Unlimited Data.
- Outcome fields — Customer_Status (Churned, Stayed, or Joined), Churn_Category, and Churn_Reason.

2.2 The Journey of the Data

The raw data begins life as a flat CSV file. It is loaded into a PostgreSQL staging table (stg_churn) exactly as-is, warts and all — including whatever nulls or inconsistencies exist in the source file. From there, it is deliberately cleaned and restructured into a production table (prod_churn), and finally split into two purpose-built views (vw_churndata and vw_joindata) that feed the Power BI dashboard. Each of these stages is covered in detail in Sections 3 through 5.

The choice of a telecom company as the business context is deliberate but not limiting: as the original tutorial notes, the same churn-analysis technique — segment customers, quantify churn by segment, identify the highest-risk combinations, and target retention effort accordingly — applies equally to retail, finance, healthcare, or any subscription-style business.

3. Data Splitting and Preprocessing

As with any ETL-driven project, "preprocessing" here means moving data through progressively cleaner and more purpose-built layers, and "splitting" means separating the customer base by outcome status so that each downstream calculation only sees the rows relevant to it.

3.1 Staging the Raw Data

The original tutorial loads the CSV into SQL Server using its GUI Import Wizard, including a workaround for a datatype quirk (changing Bit columns to Varchar(50) because the Bit type caused import errors). In PostgreSQL, the equivalent step is performed either through a GUI tool's import wizard (e.g., pgAdmin's Import/Export) or with the `\copy` command from `psql`. PostgreSQL does not have the same Bit-import quirk as SQL Server, but the same principle applies: define staging columns loosely (e.g., text/varchar) first, then enforce stricter types and constraints only after the data has been validated.

```
CREATE DATABASE db_churn;

-- Load raw CSV into a staging table (via pgAdmin import wizard or \copy)
-- \copy stg_churn FROM 'telecom_customer_data.csv' WITH (FORMAT csv, HEADER true);
```

3.2 Splitting Customers by Status

The single most important "split" in this project is not a machine-learning train/test split — it is a business-logic split on the `Customer_Status` field. Customers fall into exactly one of three states, and each state serves a different analytical purpose:

Customer_Status	Meaning	Used For
Stayed	Existing customer, still active	Churn-rate denominator; churn-driver analysis
Churned	Customer has left	Churn-rate numerator; churn-driver analysis
Joined	Newly acquired customer, no churn history yet	New Joiners metric; future churn-risk scoring

This split is implemented directly as two PostgreSQL views built on top of the cleaned production table — `vw_churndata` (Churned + Stayed, used for historical churn analysis) and `vw_joindata` (Joined only, used for new-customer tracking). This is covered fully in Section 5.

4. Pre-Analysis

Before any cleaning or transformation happens, the staging table is audited. This pre-analysis phase answers two questions: what does a healthy distribution of this data look like, and where are the gaps?

4.1 Distribution Checks

```
-- Gender distribution
SELECT stg_churn."Gender", COUNT(stg_churn."Gender") AS TotalCount,
       COUNT(stg_churn."Gender") * 100.0 / (SELECT COUNT(*) FROM stg_churn) AS Percentage
FROM stg_churn
GROUP BY stg_churn."Gender";

-- Contract type distribution
SELECT stg_churn."Contract", COUNT(stg_churn."Contract"),
       COUNT(stg_churn."Contract") * 100.0 / (SELECT COUNT(*) FROM stg_churn) AS Percentage
FROM stg_churn
GROUP BY stg_churn."Contract";

-- Revenue distribution by churn status
SELECT stg_churn."Customer_Status", COUNT(stg_churn."Customer_Status") AS TotalCount,
       SUM(stg_churn."Total_Revenue") AS TotalRev,
       SUM(stg_churn."Total_Revenue") / (SELECT SUM(stg_churn."Total_Revenue") FROM stg_churn) * 100 AS
RevPercentage
FROM stg_churn
GROUP BY stg_churn."Customer_Status";

-- State distribution, ranked
SELECT stg_churn."State", COUNT(stg_churn."State") AS TotalCount,
       COUNT(stg_churn."State") * 100.0 / (SELECT COUNT(*) FROM stg_churn) AS Percentage
FROM stg_churn
GROUP BY stg_churn."State"
ORDER BY Percentage DESC;
```

These distribution checks serve a quality-control purpose as much as an exploratory one: an unexpectedly skewed Gender or State split, for example, would be a signal to double-check the source file before trusting any downstream dashboard built on top of it.

4.2 Null Audit

A single wide query counts NULLs across every column of the staging table in one pass, using conditional SUM(CASE WHEN ... IS NULL THEN 1 ELSE 0 END) logic:

```
SELECT
  SUM(CASE WHEN stg_churn."Customer_ID" IS NULL THEN 1 ELSE 0 END) AS Customer_ID_Null_Count,
  SUM(CASE WHEN stg_churn."Value_Deal" IS NULL THEN 1 ELSE 0 END) AS Value_Deal_Null_Count,
  SUM(CASE WHEN stg_churn."Multiple_Lines" IS NULL THEN 1 ELSE 0 END) AS Multiple_Lines_Null_Count,
  SUM(CASE WHEN stg_churn."Internet_Type" IS NULL THEN 1 ELSE 0 END) AS Internet_Type_Null_Count,
  -- ... repeated for every remaining column (Online_Security, Online_Backup,
  --   Device_Protection_Plan, Premium_Support, Streaming_TV/Movies/Music,
  --   Unlimited_Data, Churn_Category, Churn_Reason, etc.)
FROM stg_churn;
```

This single query produces a one-row "null scorecard" across every column, telling the analyst exactly which fields need a COALESCE default before the data is trusted for reporting — the direct input to Section 5's cleaning step.

5. In-Analysis

With the data profiled, the in-analysis phase performs the actual cleaning, restructuring, and transformation — first in PostgreSQL, then in Power BI's Power Query editor.

5.1 Cleaning Nulls Into a Production Table

Every column flagged by the null audit is wrapped in COALESCE with a sensible default (e.g., a missing service field defaults to 'No', a missing plan defaults to 'None', a missing churn category defaults to 'Others'). The result is materialized as a new, clean table:

```
CREATE TABLE prod_churn AS (
  SELECT stg_churn."Customer_ID", stg_churn."Gender", stg_churn."Age", stg_churn."Married",
    stg_churn."State", stg_churn."Number_of_Referrals", stg_churn."Tenure_in_Months",
    COALESCE(stg_churn."Value_Deal", 'None') AS Value_Deal,
    stg_churn."Phone_Service",
    COALESCE(stg_churn."Multiple_Lines", 'No') AS Multiple_Lines,
    stg_churn."Internet_Service",
    COALESCE(stg_churn."Internet_Type", 'None') AS Internet_Type,
    COALESCE(stg_churn."Online_Security", 'No') AS Online_Security,
    COALESCE(stg_churn."Online_Backup", 'No') AS Online_Backup,
    COALESCE(stg_churn."Device_Protection_Plan", 'No') AS Device_Protection_Plan,
    COALESCE(stg_churn."Premium_Support", 'No') AS Premium_Support,
    COALESCE(stg_churn."Streaming_TV", 'No') AS Streaming_TV,
    COALESCE(stg_churn."Streaming_Movies", 'No') AS Streaming_Movies,
    COALESCE(stg_churn."Streaming_Music", 'No') AS Streaming_Music,
    COALESCE(stg_churn."Unlimited_Data", 'No') AS Unlimited_Data,
    stg_churn."Contract", stg_churn."Paperless_Billing", stg_churn."Payment_Method",
    stg_churn."Monthly_Charge", stg_churn."Total_Charges", stg_churn."Total_Refunds",
```

```

    stg_churn."Total_Extra_Data_Charges", stg_churn."Total_Long_Distance_Charges",
    stg_churn."Total_Revenue", stg_churn."Customer_Status",
    COALESCE(stg_churn."Churn_Category", 'Others') AS Churn_Category,
    COALESCE(stg_churn."Churn_Reason", 'Others') AS Churn_Reason
FROM stg_churn
);

```

5.2 Splitting Into Reporting Views

```

CREATE VIEW vw_churndata AS (
    SELECT * FROM prod_churn
    WHERE prod_churn."Customer_Status" IN ('Churned', 'Stayed')
);

CREATE VIEW vw_joindata AS (
    SELECT * FROM prod_churn
    WHERE prod_churn."Customer_Status" IN ('Joined')
);

```

5.3 Power BI Power Query Transformations

With clean views feeding Power BI, several calculated columns and reference tables are added directly in Power Query to support the dashboard's grouping logic:

- Churn Status — a 1/0 numeric flag (1 = Customer_Status is "Churned", else 0), enabling it to be summed directly by DAX measures.
- Monthly Charge Range — buckets Monthly_Charge into "< 20", "20–50", "50–100", and "> 100" bands for easier visual grouping.
- mapping_AgeGrp (reference table) — deduplicated Age values mapped into "< 20", "20–35", "36–50", and "> 50" groups, plus a numeric AgeGrpSorting column so the groups display in logical rather than alphabetical order.
- mapping_TenureGrp (reference table) — deduplicated Tenure_in_Months values mapped into "< 6 Months" through ">= 24 Months" bands, with an equivalent TenureGrpSorting column.
- prod_Services (reference table) — the individual service columns (Streaming TV, Online Security, etc.) unpivoted into a long Services / Status pair, enabling a single visual to compare churn rate across every service at once.

6. Post-Analysis and Insights

6.1 Core DAX Measures

Four DAX measures form the analytical backbone of the dashboard, each building on the last:

```

Total Customers = COUNT(prod_Churn[Customer_ID])

New Joiners = CALCULATE(COUNT(prod_Churn[Customer_ID]),
    prod_Churn[Customer_Status] = "Joined")

Total Churn = SUM(prod_Churn[Churn Status])

Churn Rate = [Total Churn] / [Total Customers]

```

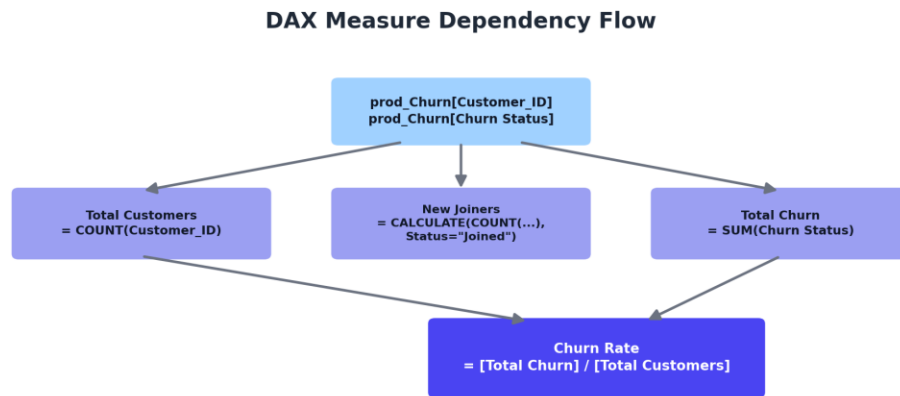


Figure 1. Dependency flow of the four core DAX measures — Churn Rate is derived from the two base measures beneath it.

6.2 Live Dashboard Output

The Summary Page shown in Figure 2 (Section 7) reflects the completed, live implementation. The headline numbers are:

Metric	Value
Total Customers	6,418
New Joiners	411
Total Churn	1,732
Churn Rate	27.0%

A churn rate of 27.0% means just over one in four customers in the base has churned — a high rate by telecom industry standards (where 15–25% annual churn is more typical), which makes the segment-level breakdown below especially important: an aggregate rate this high is rarely uniform, and the dashboard confirms it is not.

6.3 Key Insights From the Live Data

- Contract type is the single strongest churn driver visible on the dashboard: Month-to-Month customers churn at 46.5%, versus 11.0% for One Year and just 2.7% for Two Year contracts. This is roughly a 17x gap between the highest- and lowest-risk contract segments.
- Payment method shows a similar, if smaller, gradient: Mailed Check customers churn at 37.8% and Bank Withdrawal at 34.4%, both well above Credit Card customers at 14.8% — automatic, card-based payment is strongly associated with retention.
- Internet Type shows Fiber Optic customers churning at 41.1%, the highest of any service tier, ahead of Cable (25.7%) and DSL (19.4%); customers with no internet service churn at only 7.8%. Fiber customers are typically the highest-paying, most competitively targeted segment, which likely explains the gap.

- Geographically, the top 5 states by churn rate are Jammu & Kashmir (57.2%), Assam (38.1%), Jharkhand (34.5%), Chhattisgarh (30.5%), and Delhi (29.9%) — all above the 27.0% company-wide average, with Jammu & Kashmir more than double it. (Note: this confirms the underlying dataset models an India-based telecom operator, since all top-churn regions are Indian states.)
- By Churn Category, "Competitor" is by far the largest reason (761 of 1,732 churned customers, ~44%), ahead of "Attitude" (301) and "Dissatisfaction" (300); Price accounts for 196. Combined, Attitude + Dissatisfaction (601, ~35%) represent a service-experience problem roughly comparable in size to competitive pressure.
- Gender shows an imbalanced split of total churn across the two categories shown on the donut (approximately 36% / 64%) — worth confirming by hovering the visual in Power BI, since the exported image alone doesn't guarantee which slice maps to which label at a glance.

Because the Machine Learning phase of the original tutorial was intentionally excluded from this project (see Section 1.1), no predictive churn-risk score exists for the vw_jointdata population (the 411 New Joiners). That metric and view remain a descriptive count and a clean future input, not a prediction.

7. Data Visualizations & Charts

Figure 2 below is the completed, live Power BI Summary Page. Figures 3 and 4 remain structural diagrams documenting how data flows into that dashboard and how its layout was planned, for readers who want the underlying architecture rather than the rendered output.

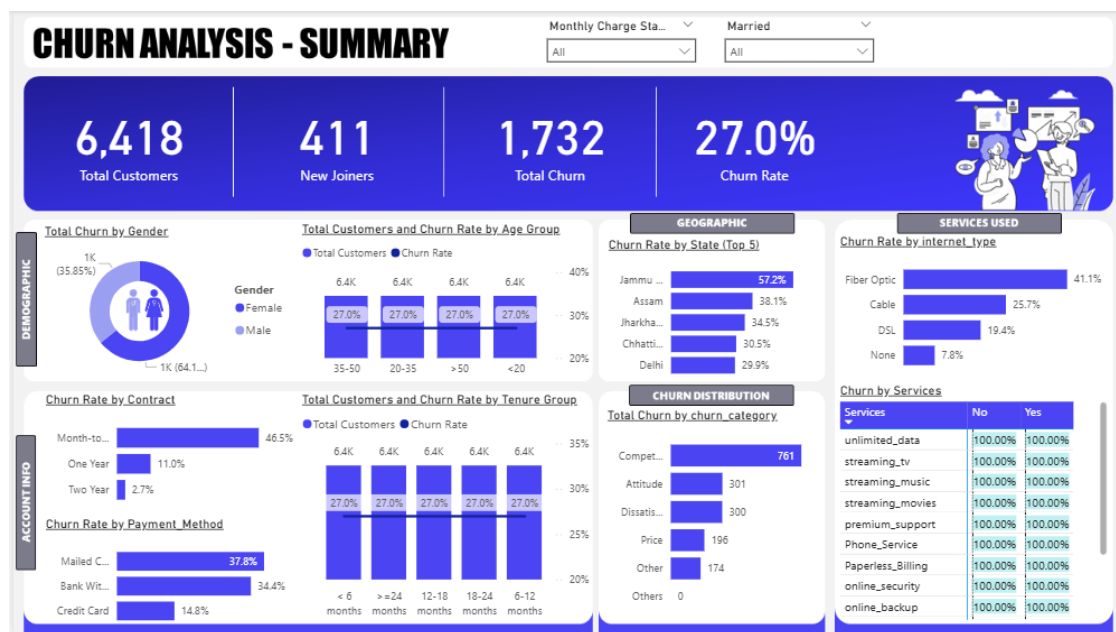
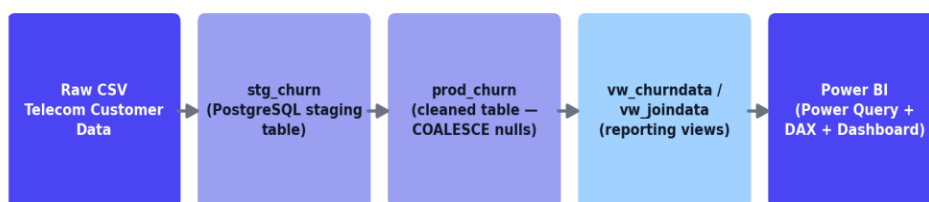


Figure 2. Live Power BI "Churn Analysis – Summary" dashboard, built on the PostgreSQL views described in Sections 3–5. Total Customers 6,418, New Joiners 411, Total Churn 1,732, Churn Rate 27.0%.

PostgreSQL ETL Pipeline → Power BI Dashboard



Adapted from a Microsoft SQL Server design to an open-source PostgreSQL stack

Figure 3. End-to-end data flow from the raw CSV through PostgreSQL staging, cleaning, and view-splitting, into the Power BI layer.

Summary Page — Dashboard Wireframe



Figure 4. Wireframe of the Power BI Summary Page, showing the KPI card row and the six analytical sections (Demographic, Account Info, Geographic, Churn Distribution, Service Used) plus the Churn Reason drill-through page.

The dashboard's visual language follows the palette specified in the project brief: #4A44F2 and #9B9FF2 as primary accent colors, #F2F2F2 as a neutral background, and #A0D1FF as a secondary highlight — the same palette used for the diagrams throughout this report.

8. Recommendations and Observations

8.1 Business Recommendations (Data-Driven)

- Prioritize contract conversion above all other levers: moving even a portion of the Month-to-Month base (46.5% churn) onto One Year or Two Year plans, via a modest discount or bundled incentive, will likely produce the single largest churn-rate improvement available, given the ~17x gap between contract types.
- Encourage a shift away from Mailed Check and Bank Withdrawal toward Credit Card / autopay billing (14.8% churn vs. 37.8% and 34.4%) — a small autopay incentive (e.g., a one-time bill credit) is inexpensive relative to the retention value it likely unlocks.
- Treat Fiber Optic (41.1% churn) as a distinct, high-priority retention segment: benchmark pricing and speeds against local competitors and consider loyalty offers targeted specifically at this tier rather than broad, undifferentiated discounts.
- Investigate Jammu & Kashmir specifically before rolling out a national retention campaign — at 57.2% churn (more than double the company average), this state likely has a distinct local driver (a competitor's aggressive entry, network quality issues, or pricing) that a one-size-fits-all fix would miss.
- Because "Competitor" is the single largest churn category (~44% of all churn), commission a competitive pricing/plan audit; because "Attitude" and "Dissatisfaction" together are nearly as large (~35%), pair that audit with customer-service quality initiatives (agent training, faster complaint resolution) — these two levers address different causes and neither alone will close the gap.

8.2 Technical & Engineering Recommendations

- Document the PostgreSQL-specific syntax differences directly in the project README for anyone following the original SQL Server tutorial alongside this implementation — most notably double-quoted, case-sensitive identifiers, native CREATE TABLE AS SELECT, and PostgreSQL's COALESCE/CASE syntax parity with SQL Server (this one translated directly with no changes needed).
- Set up a scheduled refresh (via Power BI's PostgreSQL connector and, if published to the Power BI Service, an on-premises data gateway) so the dashboard reflects new customer data automatically rather than requiring a manual re-import each time.
- Add basic data-validation constraints (e.g., NOT NULL, CHECK) to prod_churn now that the null audit and COALESCE cleaning have established what "clean" looks like, to prevent a future raw load from silently reintroducing the same nulls.
- Treat the vw_joindata view as the foundation for a future phase: if the Machine Learning portion of the original tutorial is revisited later, this view is already the correctly-shaped input population for a churn-propensity model.
- Version-control the SQL scripts and the .pbix file together (e.g., in a Git repository) so that schema changes in PostgreSQL and the corresponding Power BI transformations stay in sync over time.

8.3 Issues to Correct in the Current Dashboard

Age Group and Tenure Group charts show identical totals in every bucket: both the "Total Customers and Churn Rate by Age Group" and "by Tenure Group" visuals display the same 6.4K customers and 27.0% churn rate for every single category (<20, 20-35, 35-50, >50; and likewise across all five tenure bands). Since 6.4K and 27.0% both equal the company-wide totals, the group field is very likely not actually slicing the visual — check that mapping_AgeGrp and mapping_TenureGrp have active relationships to prod_Churn (on Age

and `Tenure_in_Months` respectively), and confirm the chart axis is bound to the mapping table's group column, not a disconnected or unrelated field.

The "Churn by Services" matrix shows 100.00% under both the No and Yes columns for every single service (unlimited_data, streaming_tv, streaming_music, streaming_movies, premium_support, phone_service, paperless_billing, online_security, online_backup). This strongly suggests the value is being shown as "% of column total" (which is trivially 100% of itself) rather than churn rate by service status. Re-check the visual's "Show value as" setting, or rebuild the measure to explicitly divide churned customers within each service/status group by total customers within that same group.

The Churn Category chart lists both "Other" (174 customers) and "Others" (0 customers) as separate categories. The 0 count confirms the COALESCE(..., 'Others') default in prod_churn (Section 5.1) is working correctly — there are no NULL Churn_Category values among churned customers — but the near-duplicate label is visually confusing. Renaming the COALESCE default to something clearly distinct, such as 'Unspecified' or 'Not Provided', would remove the ambiguity even though it isn't currently affecting the numbers.

8.4 Observations

- The three-way split of `Customer_Status` (Churned / Stayed / Joined) is doing more analytical work than it first appears — it simultaneously defines the churn-rate population, isolates new customers from that population, and sets up the future ML population, all from a single categorical column.
- Unpivoting the service columns into a single Services/Status pair (Section 5.3) is a notably efficient Power Query pattern in principle, though Section 8.3 notes the resulting matrix currently needs a fix before its numbers can be trusted.
- Because Churn Rate is defined as a ratio of two other measures rather than a raw column, it will automatically and correctly recalculate under any filter context applied in the report (by state, by contract type, etc.) without any additional DAX — the Contract and Payment Method visuals in Section 6.3 demonstrate this working as intended, which also makes the flat Age/Tenure Group results in Section 8.3 stand out as a real anomaly rather than an inherent limitation of the measure.

9. Conclusion

This project implements a complete, working ETL-to-dashboard pipeline for telecom churn analysis: raw customer data is staged, audited for nulls and distribution anomalies, cleaned into a production table, split into purpose-built reporting views, and finally surfaced through a six-section Power BI dashboard driven by four interlocking DAX measures. The substitution of PostgreSQL for the original tutorial's Microsoft SQL Server required only syntax-level adaptation — the underlying ETL logic (staging, null-auditing, COALESCE-based cleaning, view-based splitting) carried over directly, which is itself a useful demonstration of how portable good SQL design is across relational database engines.

The finished dashboard reveals a 27.0% overall churn rate driven overwhelmingly by three factors: Month-to-Month contracts, non-Credit Card payment methods, and Fiber Optic service — with Competitor pressure the single largest stated reason, closely followed by service-experience issues

(Attitude and Dissatisfaction combined). These findings, alongside the two dashboard issues flagged in Section 8.3, give this project's next owner a concrete, prioritized punch list rather than an open-ended "explore the data" starting point.

The Machine Learning component described in the original tutorial's title was intentionally left out of this implementation's scope, and this report should be read as covering the SQL/ETL and Power BI portions only. What remains is nonetheless a complete, decision-ready churn dashboard: a business user can open it and immediately see not just how much churn is occurring, but where, among whom, and — via the Churn Reason page — why.

10. References & Appendices

10.1 Reference & Credit

Original project design, dataset structure, and dashboard specification: YouTube channel Pivotal Stats, video "PowerBI End to End Churn Analysis Portfolio Project | Power BI + SQL + Machine Learning | 2024." All dashboard layout decisions, KPI definitions, and the color palette (#4A44F2, #9B9FF2, #F2F2F2, #A0D1FF) referenced in this report originate from that source material. This implementation independently substitutes PostgreSQL for Microsoft SQL Server and omits the Machine Learning phase, as stated in Section 1.1.

10.2 Appendix A — prod_churn Schema

Category	Columns
Identity / Demographic	Customer_ID, Gender, Age, Married
Geographic	State
Account	Number_of_Referrals, Tenure_in_Months, Value_Deal, Contract, Paperless_Billing, Payment_Method
Services	Phone_Service, Multiple_Lines, Internet_Service, Internet_Type, Online_Security, Online_Backup, Device_Protection_Plan, Premium_Support, Streaming_TV, Streaming_Movies, Streaming_Music, Unlimited_Data
Financial	Monthly_Charge, Total_Charges, Total_Refunds, Total_Extra_Data_Charges, Total_Long_Distance_Charges, Total_Revenue
Outcome	Customer_Status, Churn_Category, Churn_Reason

10.3 Appendix B — Full PostgreSQL Script

The complete PostgreSQL script used for staging, exploration, cleaning, and view creation is reproduced below for reproducibility.

```

CREATE DATABASE db_churn;

-- (1) Load CSV into stg_churn via pgAdmin import or \copy

-- (2) Distribution checks
SELECT stg_churn."Gender", COUNT(stg_churn."Gender") AS TotalCount,
       COUNT(stg_churn."Gender") * 100.0 / (SELECT COUNT(*) FROM stg_churn) AS Percentage
FROM stg_churn GROUP BY stg_churn."Gender";

SELECT stg_churn."Contract", COUNT(stg_churn."Contract"),
       COUNT(stg_churn."Contract") * 100.0 / (SELECT COUNT(*) FROM stg_churn) AS Percentage
FROM stg_churn GROUP BY stg_churn."Contract";

SELECT stg_churn."Customer_Status", COUNT(stg_churn."Customer_Status") AS TotalCount,
       SUM(stg_churn."Total_Revenue") AS TotalRev,
       SUM(stg_churn."Total_Revenue") / (SELECT SUM(stg_churn."Total_Revenue") FROM stg_churn) * 100 AS
RevPercentage
FROM stg_churn GROUP BY stg_churn."Customer_Status";

SELECT stg_churn."State", COUNT(stg_churn."State") AS TotalCount,
       COUNT(stg_churn."State") * 100.0 / (SELECT COUNT(*) FROM stg_churn) AS Percentage
FROM stg_churn GROUP BY stg_churn."State" ORDER BY Percentage DESC;

-- (3) Null audit (abridged – repeat pattern for every column)
SELECT
    SUM(CASE WHEN stg_churn."Customer_ID" IS NULL THEN 1 ELSE 0 END) AS Customer_ID_Null_Count,
    SUM(CASE WHEN stg_churn."Churn_Reason" IS NULL THEN 1 ELSE 0 END) AS Churn_Reason_Null_Count
FROM stg_churn;

-- (4) Clean into production table
CREATE TABLE prod_churn AS (
    SELECT stg_churn."Customer_ID", stg_churn."Gender", stg_churn."Age", stg_churn."Married",
           stg_churn."State", stg_churn."Number_of_Referrals", stg_churn."Tenure_in_Months",
           COALESCE(stg_churn."Value_Deal", 'None') AS Value_Deal,
           stg_churn."Phone_Service",
           COALESCE(stg_churn."Multiple_Lines", 'No') AS Multiple_Lines,
           stg_churn."Internet_Service",
           COALESCE(stg_churn."Internet_Type", 'None') AS Internet_Type,
           COALESCE(stg_churn."Online_Security", 'No') AS Online_Security,
           COALESCE(stg_churn."Online_Backup", 'No') AS Online_Backup,
           COALESCE(stg_churn."Device_Protection_Plan", 'No') AS Device_Protection_Plan,
           COALESCE(stg_churn."Premium_Support", 'No') AS Premium_Support,
           COALESCE(stg_churn."Streaming_TV", 'No') AS Streaming_TV,
           COALESCE(stg_churn."Streaming_Movies", 'No') AS Streaming_Movies,
           COALESCE(stg_churn."Streaming_Music", 'No') AS Streaming_Music,
           COALESCE(stg_churn."Unlimited_Data", 'No') AS Unlimited_Data,
           stg_churn."Contract", stg_churn."Paperless_Billing", stg_churn."Payment_Method",
           stg_churn."Monthly_Charge", stg_churn."Total_Charges", stg_churn."Total_Refunds",
           stg_churn."Total_Extra_Data_Charges", stg_churn."Total_Long_Distance_Charges",
           stg_churn."Total_Revenue", stg_churn."Customer_Status",
           COALESCE(stg_churn."Churn_Category", 'Others') AS Churn_Category,
           COALESCE(stg_churn."Churn_Reason", 'Others') AS Churn_Reason
    FROM stg_churn
);

-- (5) Reporting views
CREATE VIEW vw_churndata AS (
    SELECT * FROM prod_churn WHERE prod_churn."Customer_Status" IN ('Churned', 'Stayed')
);
CREATE VIEW vw_joindata AS (
    SELECT * FROM prod_churn WHERE prod_churn."Customer_Status" IN ('Joined')
);

```

10.4 Appendix C — Power BI DAX Measures

```
Total Customers = COUNT(prod_Churn[Customer_ID])  
New Joiners = CALCULATE(COUNT(prod_Churn[Customer_ID]), prod_Churn[Customer_Status] = "Joined")  
Total Churn = SUM(prod_Churn[Churn_Status])  
Churn Rate = [Total Churn] / [Total Customers]
```